



COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Priority Queues – Implementation 1



Agenda & Reading

► Agenda

- Implementing a Priority Queue using a binary heap
- Binary heap properties
- PriorityQueue class
 - Constructor
 - insert()



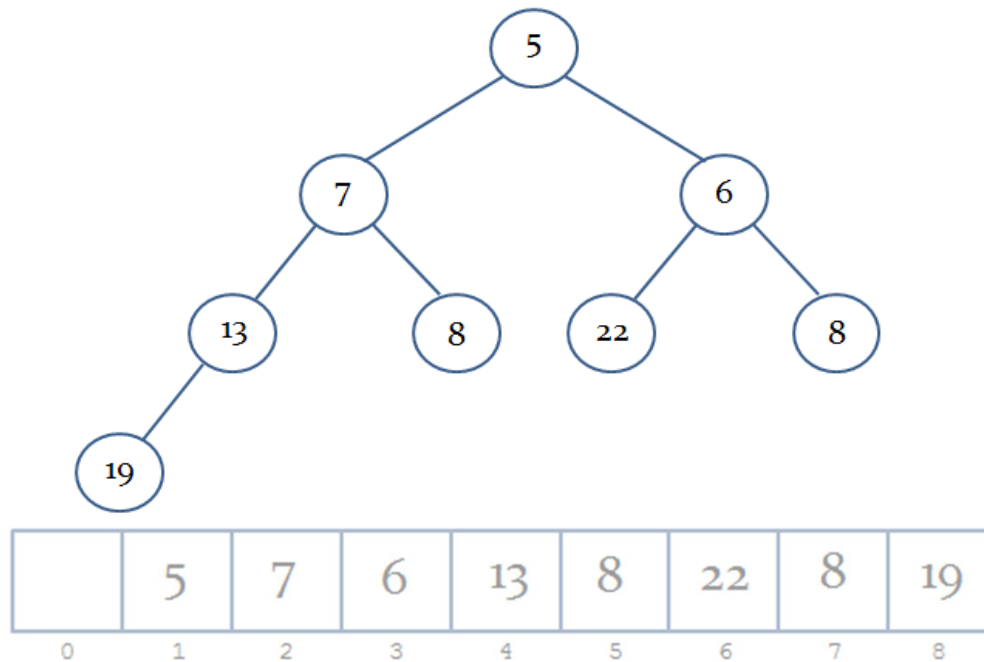
► Reading

- Online Textbook: <https://onlinetextbook.auckland.ac.nz/20-priority-queues-and-heaps/>



Binary heap

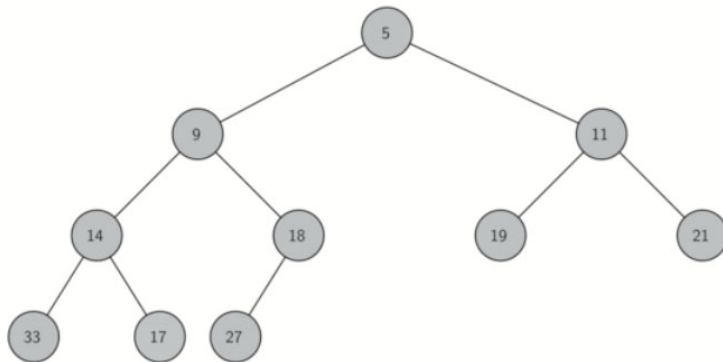
- The classic way to implement a priority queue is with a **binary heap**





Binary heap

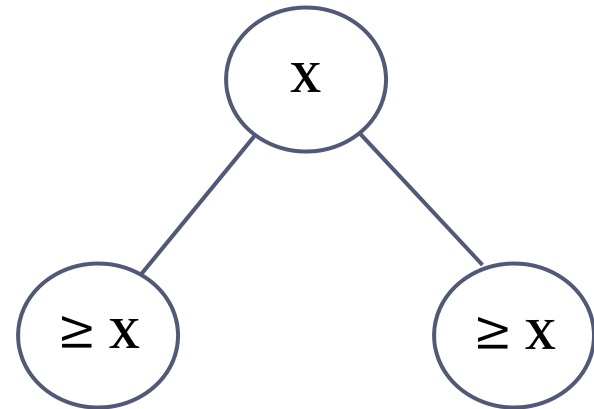
- ▶ The classic way to implement a priority queue is with a **binary heap**
- ▶ A binary heap has two special properties:



a *complete* binary tree

Each level has all possible nodes, except for the bottom level which is filled from left to right

shape property



a **partial** ordering over nodes
the key at every parent node is less than or equal to **both** of its **children**

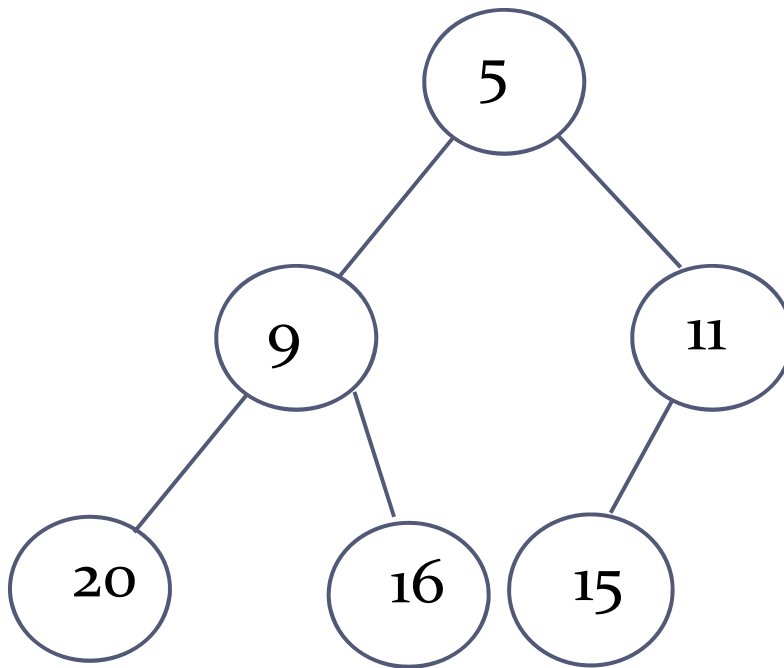
order property



Binary heaps

► *Question 1: Is the following a Binary Heap?*

Yes

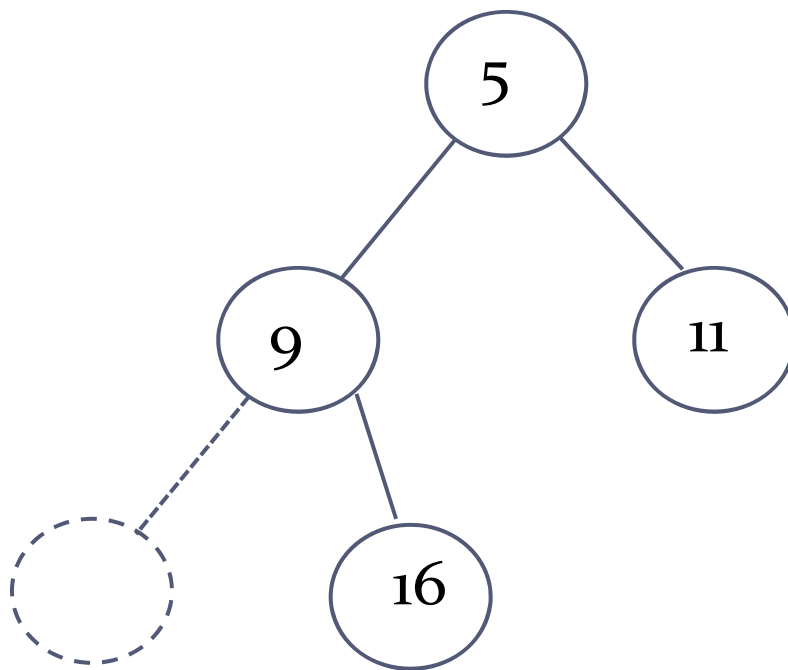




Binary heaps

► Question 2: *Is the following a Binary Heap?*

No

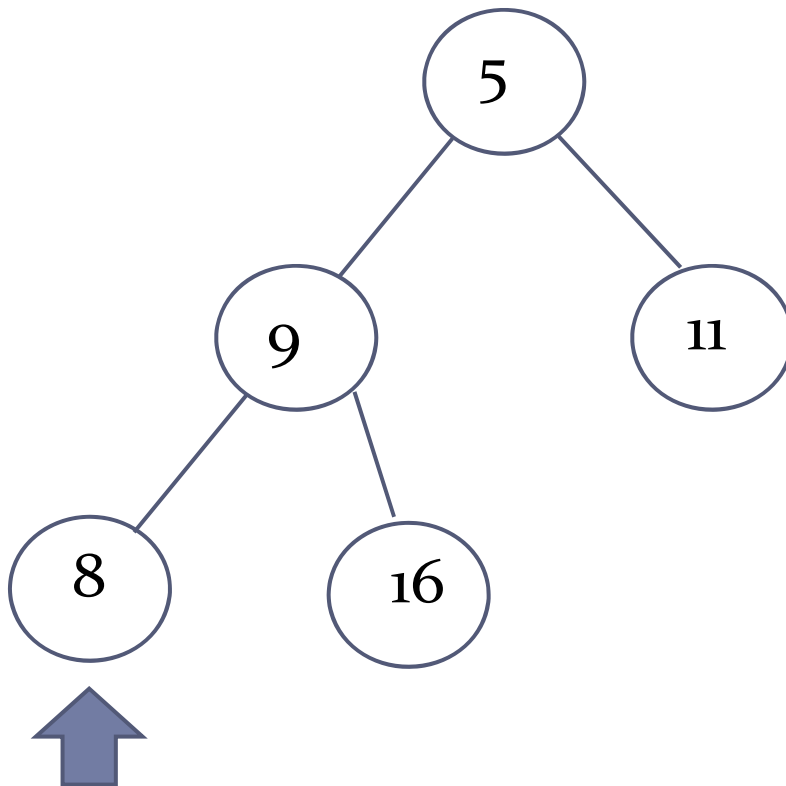




Binary heaps

► Question 3: *Is the following a Binary Heap?*

No

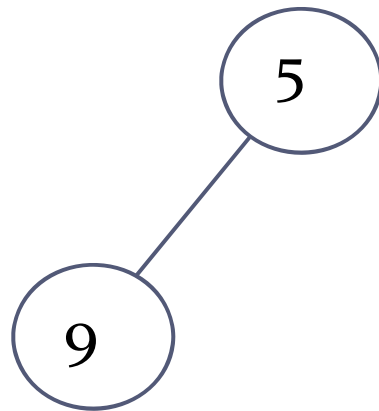




Binary heaps

► Question 4: *Is the following a Binary Heap?*

Yes





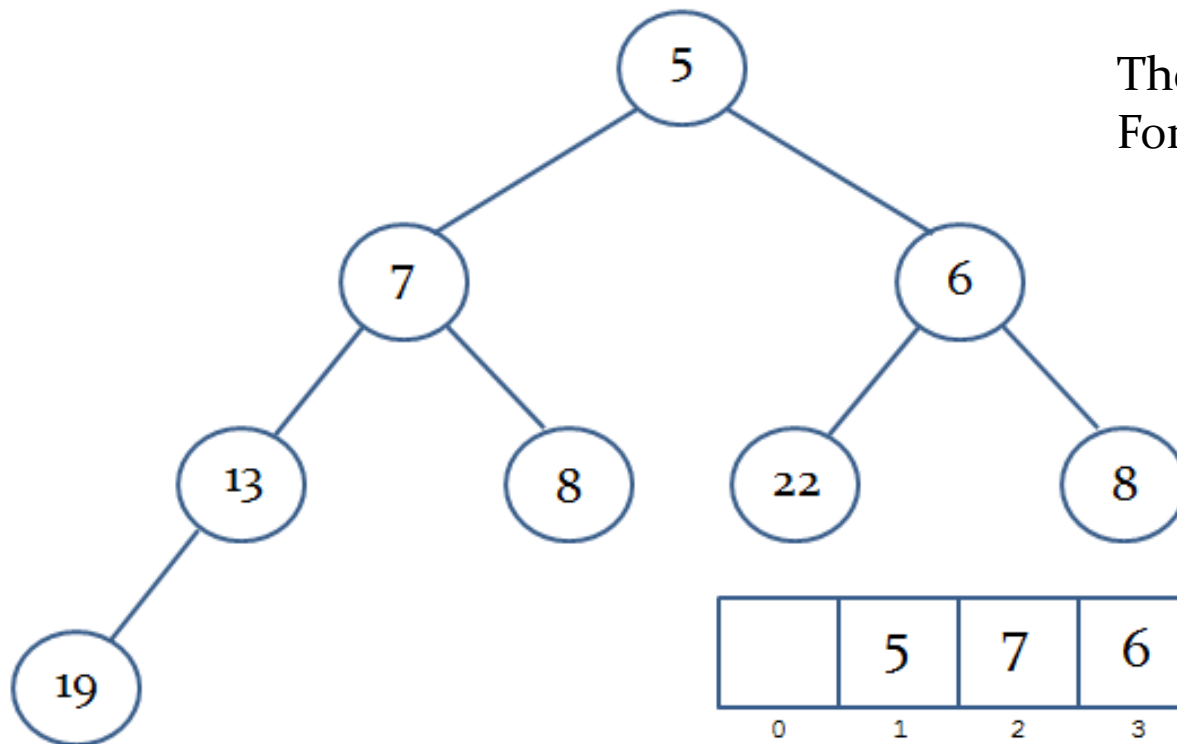
Complete Binary Trees

- ▶ Things to note about complete binary trees:
 - ▶ they are balanced (i.e., the height of the left and right subtree differs by at most 1)
 - ▶ half the nodes are leaves (or $\text{half} + 1$)
 - ▶ the left subtree always has more or the same number of nodes as the right subtree.



Binary heap

- ▶ The classic way to implement a priority queue is with a **binary heap**
- ▶ We can use a **Python list** to implement a binary heap:



The root is stored at index 1

For a node at index position i :

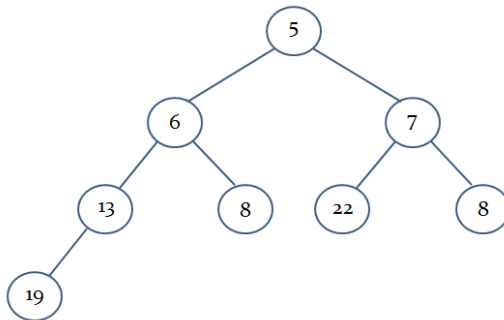
- **left child** at index $2i$
- **right child** at index $2i+1$
- **parent (if not root)** at index $i//2$ (integer division)

	5	7	6	13	8	22	8	19
0	1	2	3	4	5	6	7	8

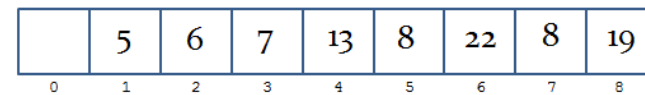


Binary heap

- ▶ The classic way to implement a priority queue is with a **binary heap**



When we draw / visualise a binary heap, we will draw a binary tree



The actual internal representation (in our code) will be a list



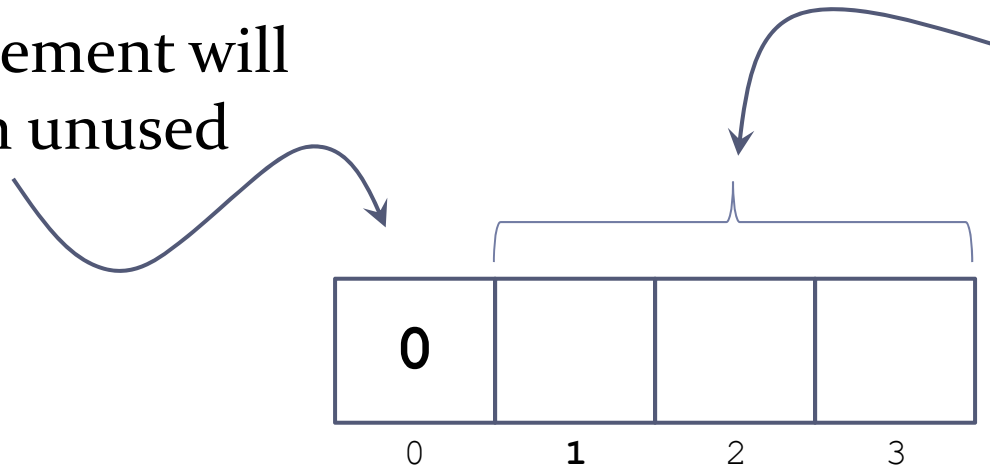
Heap operations

- ▶ Creating a new, empty heap:

```
class PriorityQueue:
```

```
    def __init__(self):  
        self.binary_heap = [0]  
        self.size = 0
```

This element will
remain unused



Our actual heap
elements will be
stored starting at
index 1



Heap operations

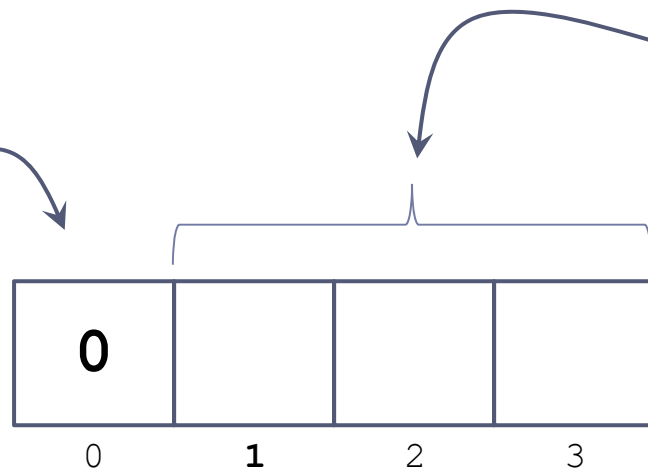
► Creating a new, empty heap:

```
class PriorityQueue:

    def __init__(self):
        self.binary_heap = [0]
        self.size = 0

    def __str__(self):
        return str(self.binary_heap)
```

This element will remain unused



Our actual heap elements will be stored starting at index 1

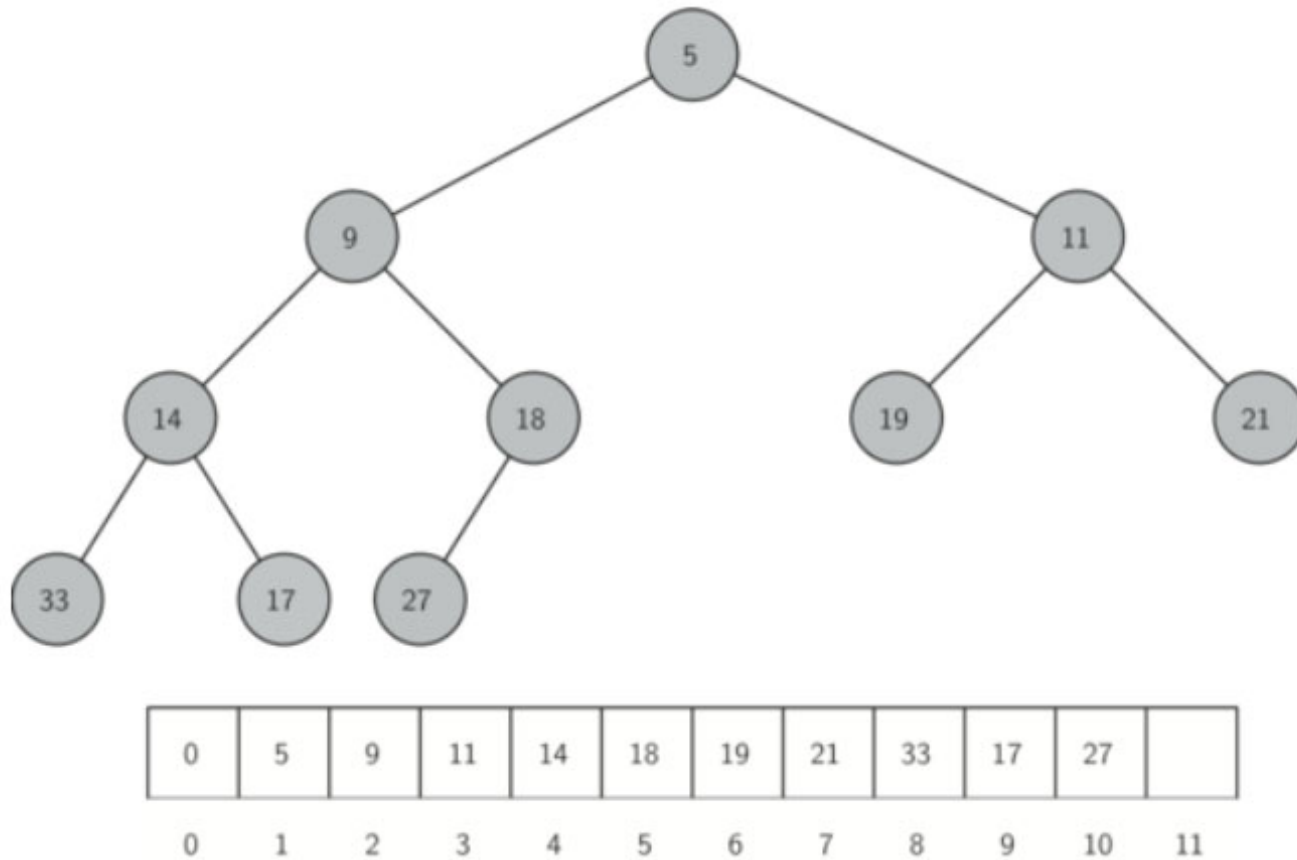
size stores the number of elements in the heap!



Heap operations – insert()

- ▶ Let's look at the important heap operations (**insert** and **delete_minimum**) in the context of the following heap

Insertion: `priority_queue.insert(7)`

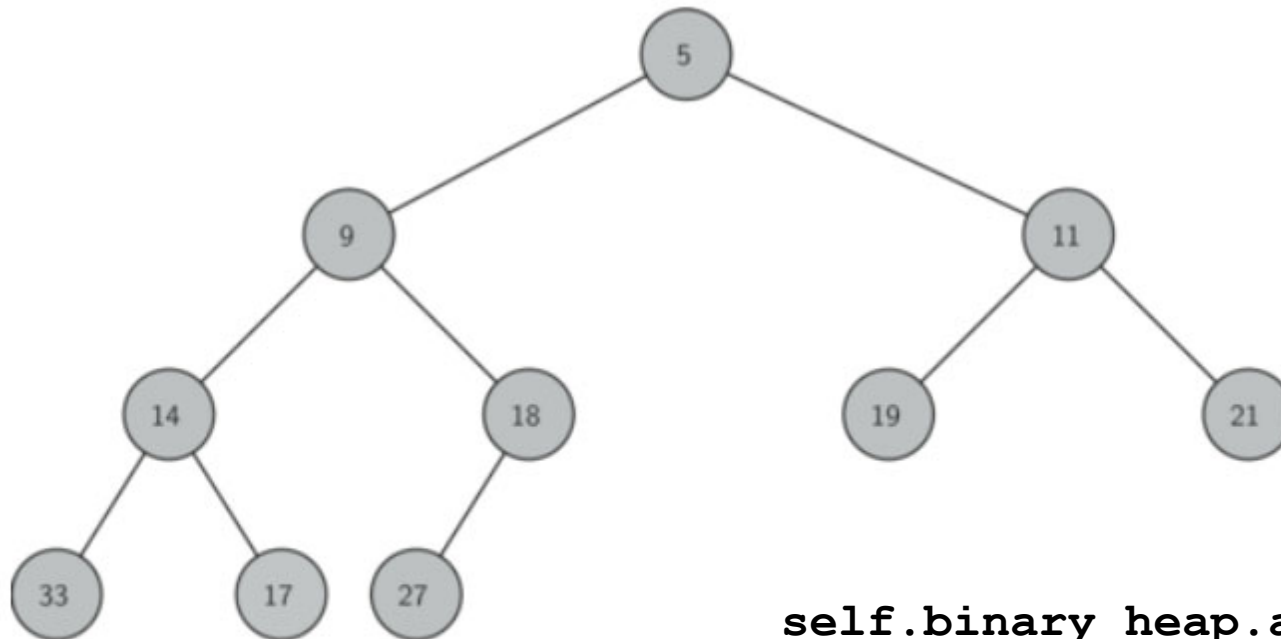




Heap operations – insert()

- ▶ Let's look at the important heap operations (**insert** and **delete_minimum**) in the context of the following heap

Insertion: `priority_queue.insert(7)`



`self.binary_heap.append(7)`

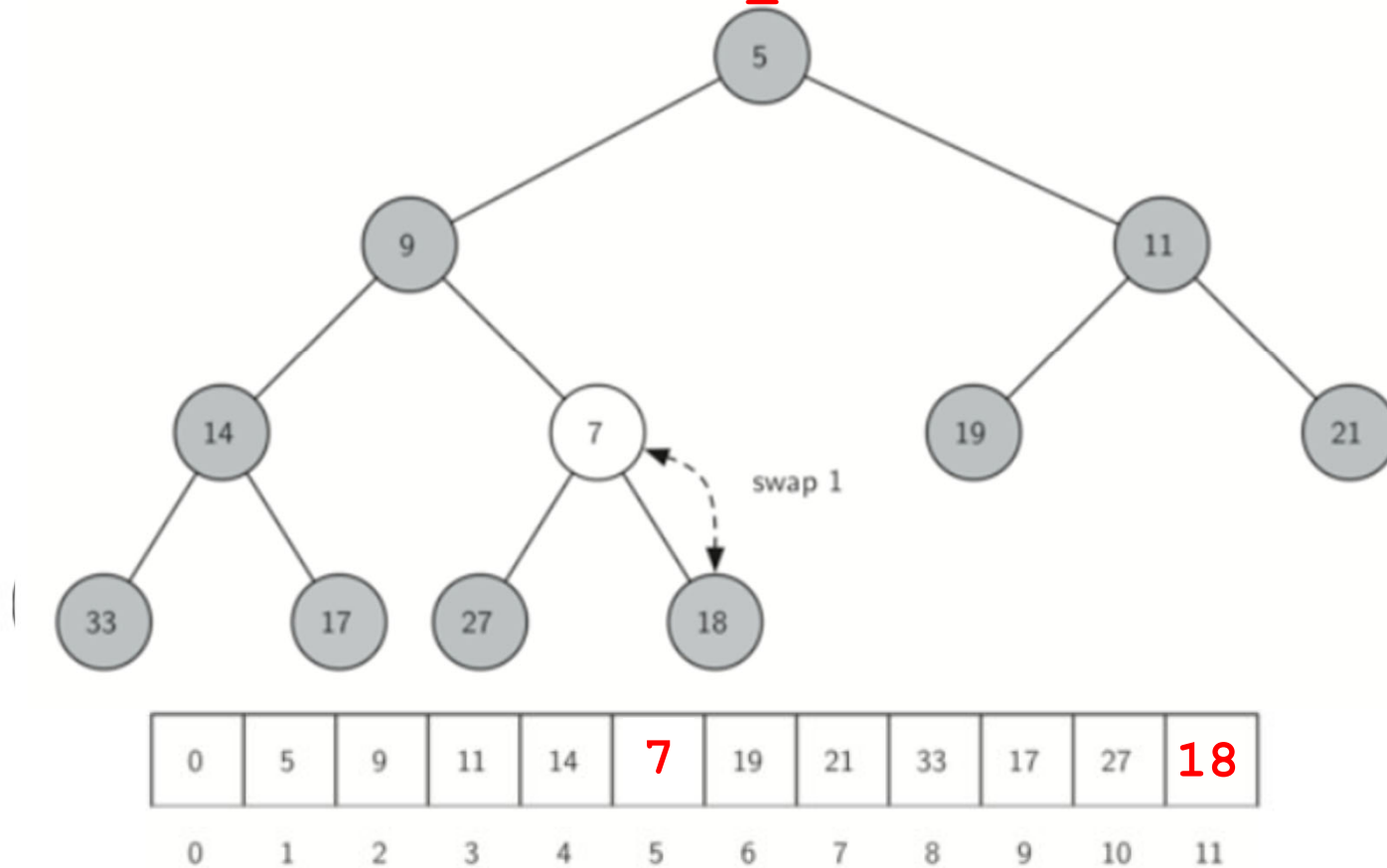
0	5	9	11	14	18	19	21	33	17	27	7
0	1	2	3	4	5	6	7	8	9	10	11



Heap operations – insert()

- Let's look at the important heap operations (**insert** and **delete_minimum**) in the context of the following heap

Insertion: `priority_queue.insert(7)`

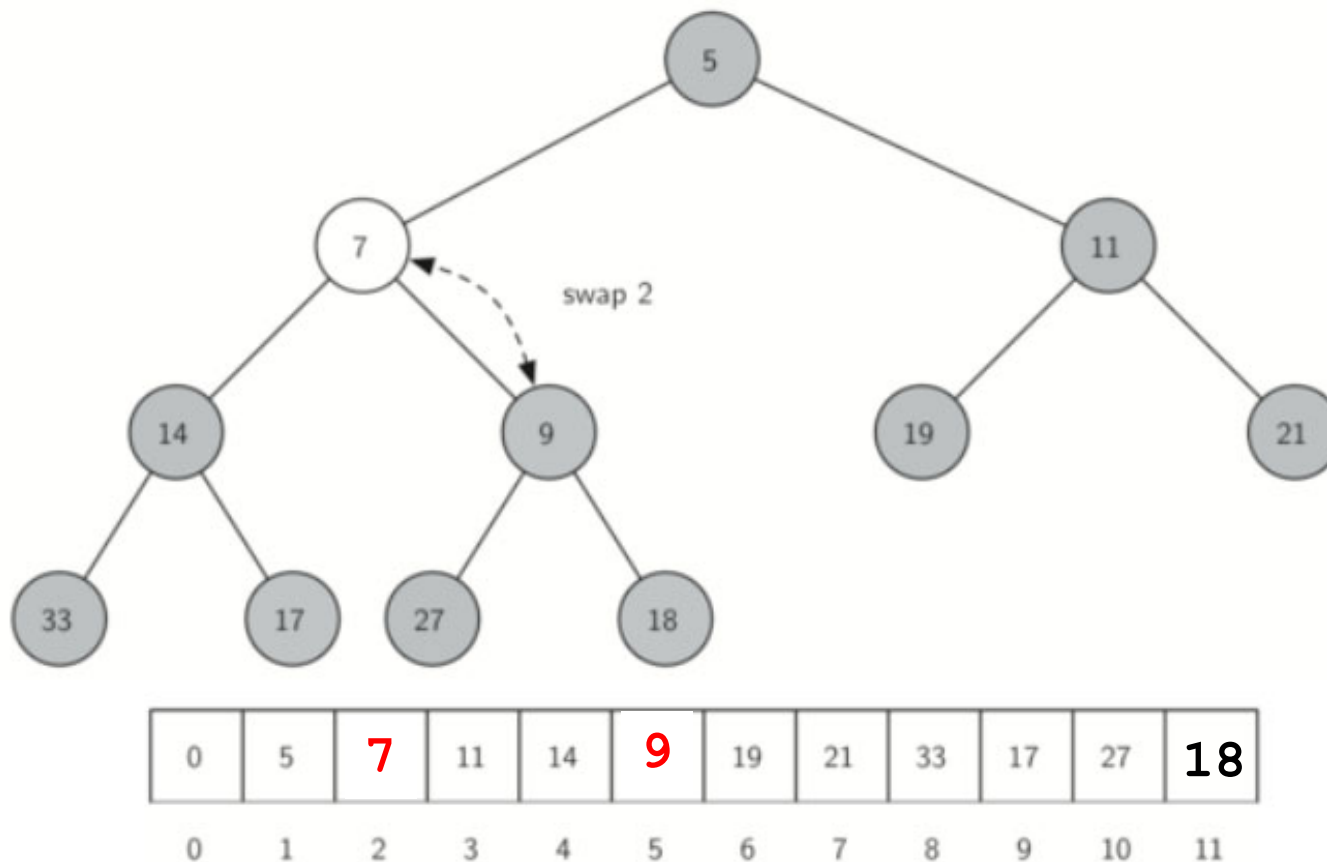




Heap operations – insert()

- Let's look at the important heap operations (**insert** and **delete_minimum**) in the context of the following heap

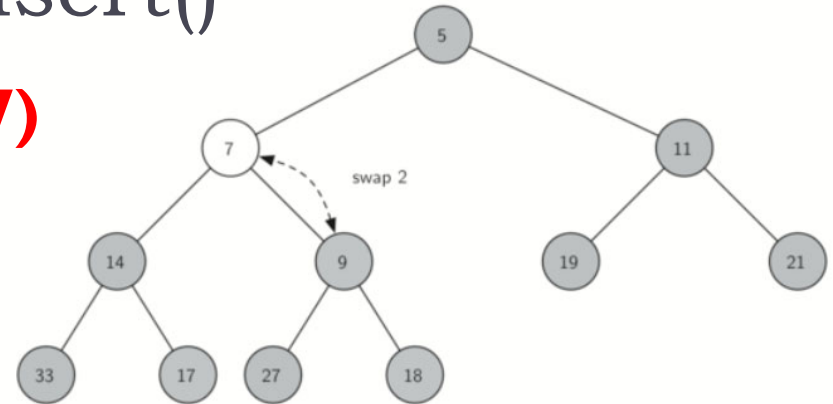
Insertion: `priority_queue.insert(7)`





Heap operations – insert()

- Insertion: **priority_queue.insert(7)**

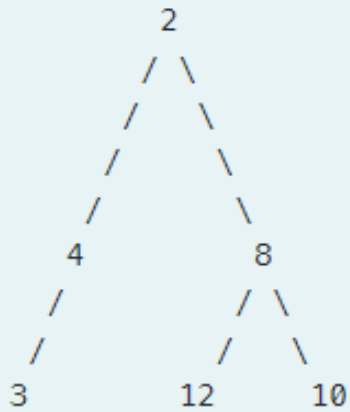


```
def percolate_up(self, index):  
    while index // 2 > 0:  
        // complete this  
  
        index = index // 2  
  
def insert(self, value):  
    self.binary_heap.append(value)  
    self.size += 1  
    self.percolate_up(self.size)
```

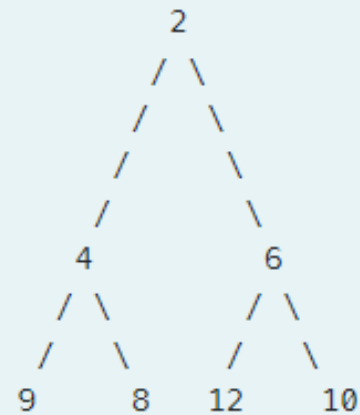


Exercise 1-2

Is the following a Binary Heap (minimum-heap)?



Is the following a Binary Heap (minimum-heap)?





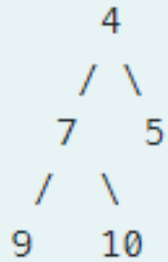
Exercise 3

- ▶ Draw the minimum-heap **represented** by the list [0, 21, 35, 48, 87, 61, 55, 73] as a tree:



Exercise 4

Given is the binary heap (a minimum-heap) in the picture below.



- ▶ What does the heap look like after inserting the value "2" into it (using the method presented in the lecture)?



Exercise 5

- ▶ Consider the following Python list:
 - ▶ `[30, 20, 15, 12, 13, 18]`
- ▶ Suppose the elements in the above list are inserted into an initially empty minimum-heap in that order. What does the binary heap look like?

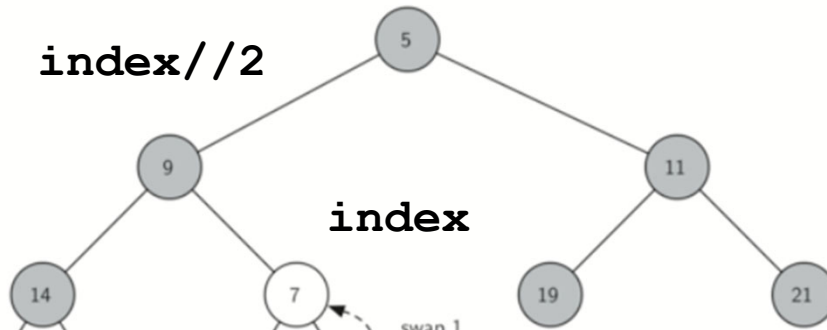


Exercise 6

► Consider the following class definition:

► Complete the `percolate_up` method

```
class PriorityQueue:
    def __init__(self):
        self.binary_heap = [0]
        self.size = 0
    def insert(self, item):
        self.binary_heap.append(item)
        self.size += 1
        self.percolate_up(self.size)
```



```
def percolate_up(self, index):
    while index // 2 > 0 and self.binary_heap[index]... :
        // swap two elements

    index = index // 2
```



Exercise 7

- ▶ Continuing on from the previous question, add the following methods to the **PriorityQueue**
 - ▶ The `__len__(self)` method which returns the number of elements present in a priority queue.
 - ▶ The `is_empty(self)` method which returns `True` if the priority queue is empty, return `False` otherwise

```
class PriorityQueue:
    def __init__(self):
        self.binary_heap = [0]
        self.size = 0
    def __str__(self):
        return str(self.binary_heap)
    def insert(self, item):
        self.binary_heap.append(item)
        self.size += 1
        self.percolate_up(self.size)
```